

SIMATS ENGINEERING

ASSESSMENT TOOL 1 — ANSWER SHEET

Course: Artificial Intelligence (CSA1711) | Annexure A: Analytical Problem Solving

SHABARISH N

192424007

Question 1 — Water Jug Problem

Problem: A 4-gallon jug and a 3-gallon jug, neither marked. Get exactly 2 gallons into the 4-gallon jug.

State representation: (x, y) where x = water in the 4-gallon jug, y = water in the 3-gallon jug

- Start state: (0, 0)

Step	Action	4-gal jug (x)	3-gal jug (y)
0	Initial state	0	0
1	Fill the 4-gallon jug from the pump	4	0
2	Pour water from the 4-gallon jug into the 3-gallon jug	1	3
3	Empty the 3-gallon jug onto the ground	1	0
4	Pour the remaining 1 gallon from the 4-gallon jug into the 3-gallon jug	0	1
5	Fill the 4-gallon jug from the pump	4	1
6	Pour from the 4-gallon jug into the 3-gallon jug until the 3-gallon jug is full (it needs 2 more gallons)	2	3

Result: After Step 6, the 4-gallon jug contains exactly 2 gallons of water — the goal state (2, 3) is reached.

Question 2 — Mars Rover Agent Analysis

(i) Percepts

- Camera images/video of the Martian terrain and rock/soil samples
- Spectrometer and chemical-analysis readings of soil/rock composition
- GPS/positional and orientation data
- Obstacle-distance and terrain-slope sensor readings
- Temperature, atmospheric pressure and weather data
- Battery/solar-power level readings
- Commands/instructions received from mission control on Earth

(ii) Environment characterization

- Partially observable — the rover cannot sense the entire Martian terrain at once
- Stochastic — outcomes of actions (e.g., wheel slip on loose soil) are not fully predictable
- Sequential — current decisions affect future states (path taken, power used)
- Dynamic — weather, lighting and terrain conditions change over time

- Continuous — position, sensor readings and time are continuous-valued
- Unknown — the exact terrain map is not known in advance and must be discovered
- Single-agent — one rover operating in the environment (not competing with other agents)

(iii) Actions the agent can take

- Move forward/backward, turn left/right
- Capture images/photographs of the surroundings
- Extend/operate the robotic arm to collect a soil or rock sample
- Drill into rock and analyse samples using onboard instruments
- Avoid or navigate around detected obstacles
- Transmit collected data to the orbiter or directly to Earth
- Adjust solar panel orientation to conserve/harvest power
- Enter low-power/idle mode when battery is low

(iv) Performance evaluation

- Distance covered safely without collisions or getting stuck
- Number and quality/accuracy of samples collected and analysed
- Accuracy and completeness of data successfully transmitted to Earth
- Energy efficiency — power consumed relative to tasks completed
- Time taken to reach designated target sites
- Longevity/durability — avoidance of damage to the rover

(v) Suitable agent architecture

A hybrid model-based, goal-based (utility-based) agent is most suitable. Because the Martian environment is partially observable, dynamic and stochastic, the rover must maintain an internal model of the world (map of explored terrain, known obstacles, current state) — this rules out a simple reflex agent, which cannot handle unseen aspects of the environment. On top of this internal model, a goal-based/utility-based layer plans efficient paths to sample-collection targets while weighing trade-offs like time, safety and power usage. A reflex layer is still used for fast, low-level responses such as immediate obstacle avoidance. This layered (hybrid) architecture gives the rover both long-term planning ability and short-term reactive safety.

Question 3 — 8-Queens Problem Formulation

Goal: Place 8 queens on an 8×8 chessboard so that no two queens attack each other (no two queens share a row, column, or diagonal).

Problem formulation (incremental/column-by-column formulation)

- Initial state: An empty chessboard (no queens placed), or equivalently, 0 queens placed.
- States: Any arrangement of 0 to 8 queens on the board, with at most one queen per column, where no placed queen attacks another.
- Actions/Successor function: Add a queen to any square in the leftmost empty column such that it is not attacked by any queen already placed.
- Goal test: All 8 queens are placed on the board and no two queens attack each other.
- Path cost: A constant cost of 1 per move (each queen placement); since only the final configuration matters, path cost is not significant here — only the goal state matters.

Why this formulation is efficient

This incremental formulation reduces the search space drastically. A naive formulation (choosing any of 64 squares for each of 8 queens) gives $64 \times 63 \times \dots \times 57 \approx 1.8 \times 10^{14}$ states. Restricting placement to one queen per column reduces this to $8^8 = 16,777,216$ states, and rejecting attacking placements as soon as they occur (as in the successor function above) prunes the space to just 2,057 possible sequences to check, making the search practically feasible using uninformed search strategies such as backtracking/depth-first search.

Note on the given figure: The example configuration shown (one queen in each column) is not a solution because the queen in the first column lies on the same diagonal as the queen in the last column — illustrating that diagonal attacks, not just row/column attacks, must be checked by the goal test.

Question 4 — OLA Cab Booking Agent

Type of problem-solving agent

This scenario fits a Goal-Based Agent that uses search-based problem solving to find a route from the pickup location to the destination. The agent's goal is 'reach the destination location'; it searches through the road/route network (a graph of locations connected by roads) to find a suitable path, and separately selects among cab-type options (mini, micro, sedan, shared, prime) based on the customer's stated preference/comfort constraint. Because the environment (traffic, road network) is only partially known and dynamic, a goal-based agent that re-plans as needed is more appropriate than a simple reflex agent.

Problem formulation

- Initial state: Customer's current pickup location
- Goal state: Customer's specified destination location
- Actions: Move from one location/junction to an adjacent one along a road segment
- Path cost: Function of distance, estimated travel time, and fare (varies by cab type)

Pseudocode

```
BEGIN OLA_CAB_BOOKING_AGENT  INPUT: pickup_location, destination_location,
preferred_cab_type           current_state <- pickup_location  goal_state <-
destination_location         available_cabs <- FIND_NEARBY_CABS(pickup_location,
preferred_cab_type)          IF available_cabs is EMPTY THEN      DISPLAY "No cabs
available for the selected type"      RETURN FAILURE  END IF      FOR EACH cab IN
available_cabs DO              cab.route <- SEARCH_ALGORITHM(cab.location, current_state)
// e.g., A*/Dijkstra            cab.eta <- COMPUTE_TIME(cab.route)  END FOR
selected_cab <- SELECT_BEST(available_cabs, criteria = [eta, rating, fare])
trip_route <- SEARCH_ALGORITHM(current_state, goal_state)  estimated_fare <-
CALCULATE_FARE(trip_route, preferred_cab_type)  estimated_time <-
CALCULATE_TIME(trip_route)  DISPLAY selected_cab, trip_route, estimated_fare,
estimated_time  IF customer_confirms_booking THEN      BOOK(selected_cab,
pickup_location, destination_location)      TRACK(selected_cab, trip_route)
RETURN SUCCESS  ELSE      RETURN CANCELLED  END IF  END
```

Question 5 — Uniform Cost Search (Logistics Network)

Graph edges (warehouse network): S–A = 1, A–B = 3, A–C = 1, B–D = 3, C–D = 1, C–G = 2, D–G = 3, S–G = 12 (direct).

UCS Execution Trace

Step	Node expanded	Frontier (node : cumulative cost)
1	S (cost 0)	A:1, G:12
2	A (cost 1)	C:2, B:4, G:12
3	C (cost 2)	D:3, G:4 (via C), B:4, G:12
4	D (cost 3)	G:4 (via C), B:4, G:6 (via D), G:12
5	G (cost 4) — Goal reached!	—

Result

Least-cost path: $S \rightarrow A \rightarrow C \rightarrow G$

Total cost = 1 (S–A) + 1 (A–C) + 2 (C–G) = 4

This is confirmed optimal against the alternatives: S–A–C–D–G = 1+1+1+3 = 6; S–A–B–D–G = 1+3+3+3 = 10; S–G (direct) = 12. UCS always expands the lowest cumulative-cost node first, guaranteeing the path found when the goal is popped from the frontier is the least-cost path.